Reviewer Pack — Minimal Hodge Diamond Diameter and Resolution Proof Width In…

Entry c01445a83aa1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 02:54:14 UTC

Minimal Hodge Diamond Diameter and Resolution Proof Width Inequality

Entry ID: `c01445a83aa1` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 02:54:14 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Hodge Theory) **Field B** (complexity object): Complexity Theory (Resolution Proof Complexity)

Statement:

> For every CNF φ with n variables, the minimal Hodge diamond diameter (hdd(φ)) of its associated algebraic variety V(φ) is linearly correlated with its resolution proof width w(φ), such that hdd(φ) = Ω(w(φ)).

Rationale (proposer's reasoning):

> The Hodge diamond is a combinatorial invariant in algebraic geometry that captures information about the dimensions and degrees of various subvarieties. If the Hodge diamond diameter grows with the resolution proof width, it suggests that there might be a geometric interpretation of complexity, potentially leading to new proof techniques for resolving P vs NP.

Taxonomy category: `Resolution Proof Complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `34cbd3428678033c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all CNFs φ with n ≤ 40 variables, the ratio hdd(φ)/w(φ) is greater than or equal to 1 across at least 80% of the seeds, and there is no seed where w(φ)/hdd(φ) exceeds 2.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Hodge diamond diameter" AND "resolution proof width" - "algebraic geometry" AND "complexity theory" AND CNF - "minimal Hodge diamond diameter" AND resolution AND complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 20  # Fixed size for simplicity, adjust as needed
    clauses = []
    for _ in range(n):
        clause = [random.randint(1, n) if random.choice([True, False]) else -random.randint(1, n)
        clauses.append(clause)

    # Construct the associated algebraic variety using Gröbner bases
    # This is a simplified version and not actual computation of Hodge diamond
    hdd = sum(abs(c) for c in clauses)  # Placeholder for actual HODGDE calculation

    # Compute resolution proof width (simplified)
    w = len(clauses) * n  # Placeholder for actual width calculation

    return {
        "metric_name": "hdd_over_w",
        "metric_value": hdd / w if w != 0 else float('inf'),
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": hdd >= w,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] if len(sys.argv) > 1 else [2**i + 1 for i in range(

    results = []
    for seed in seeds:
        result = run_trial(seed)
```

```
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fractior
    elif sum(1 for r in results if not r["conjecture_holds"]) / len(results) >= 0.2:
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed=0")
    else:
        print(f"RESULT: INCONCLUSIVE reason=\"insufficient_data\" n_tested={len(seeds)}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4b83d262.py", line 47, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4b83d262.py", line 28, in run_trial
    hdd = sum(abs(c) for c in clauses)  # Placeholder for actual HODGDE calculation
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4b83d262.py", line 28, in <genexpr>
    hdd = sum(abs(c) for c in clauses)  # Placeholder for actual HODGDE calculation
                ^^^^^^
TypeError: bad operand type for abs(): 'list'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from verifying the conjecture's conditions. | next: Review and debug the test code to ensure it can run successfully and produce results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12453
2	preregistration	ollama_remote	glm4:latest	0	0	9495
3	novelty	ollama_remote	glm4:latest	0	0	10804
4	novelty	ollama_remote	glm4:latest	0	0	9467
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15879
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10186
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7390
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	6249
9	judge	ollama_remote	glm4:latest	0	0	23431

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 105354 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/c01445a83aa1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/c01445a83aa1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/c01445a83aa1.tar.gz` (if generated)